

Étude du chiffrement totalement homomorphe

Fonctionnement de DGHV, CKKS et ouverture vers TFHE

Constantin Delahodde Martin Chedozeau Lucas Schaeffer

Licence Double Diplôme Mathématiques–Physique
Université de Versailles Saint-Quentin

Sous la direction de M. Jacques Patarin

Année universitaire 2025–2026

Objectif de la présentation

Question centrale

Comment peut-on calculer sur des données chiffrées sans jamais les déchiffrer ?

- Comprendre le principe général du FHE.
- Expliquer le rôle du bruit.
- Étudier les étapes algorithmiques des schémas.

Plan

- 1 Modèle de calcul du FHE.
- 2 DGHV : chiffrement sur les entiers.
- 3 CKKS : calcul approché vectorisé.
- 4 TFHE : calcul booléen rapide.

Le verrou que le FHE cherche à lever

Cryptographie classique

Une donnée est soit protégée, soit utilisable directement. Pour calculer dessus, il faut généralement la déchiffrer.

$$m \xrightarrow{\text{Enc}} c \quad c \xrightarrow{\text{Dec}} m \quad \text{calcul en clair : } f(m)$$

Idée du FHE

Déléguer le calcul à un serveur qui manipule seulement des chiffrés :

$$\text{Eval}(f, \text{Enc}(m_1), \dots, \text{Enc}(m_k)) = \text{Enc}(f(m_1, \dots, m_k)).$$

Pourquoi addition et multiplication suffisent

Circuits booléens

Un programme peut être vu comme un circuit composé de portes logiques.

$$\text{XOR} \leftrightarrow + \bmod 2, \quad \text{AND} \leftrightarrow \times \bmod 2.$$

Critère de succès

Si un schéma permet d'additionner et de multiplier les chiffrés, alors il peut évaluer tout circuit :

$$\text{Dec}(c_1 + c_2) = m_1 \oplus m_2,$$

$$\text{Dec}(c_1 c_2) = m_1 m_2.$$

Le bruit : sécurité et limite du calcul

Principe commun aux schémas étudiés

Le bruit empêche de retrouver la clé ou le message à partir des chiffrés. Mais chaque calcul augmente ce bruit.

- bruit faible : le déchiffrement reste correct ;
- bruit trop grand : le message est détruit ;
- multiplication : l'opération la plus coûteuse.

Bootstrapping

Évaluer homomorphiquement le déchiffrement pour obtenir un nouveau chiffré du même message, mais avec moins de bruit.

-
- 1978** Rivest, Adleman et Dertouzos introduisent les *privacy homomorphisms*.
 - 2009** Gentry construit le premier FHE grâce au bootstrapping.
 - 2010** DGHV simplifie l'approche avec des entiers et le problème AGCD.
 - 2016** TFHE accélère le bootstrapping sur le tore.
 - 2017** CKKS propose un FHE approché pour les réels et les complexes.
-

Fil du mémoire

Les trois schémas répondent au même problème, mais avec des choix différents : exactitude, approximation, calcul logique, vectorisation, coût du bootstrapping.

DGHV : pourquoi commencer par ce schéma ?

Intérêt pédagogique

DGHV remplace les réseaux de Gentry par de l'arithmétique sur les entiers. Cela rend très visible le mécanisme central : bruit, seuil critique, profondeur.

- Message : un bit $b \in \{0, 1\}$.
- Clé secrète : un grand entier impair p .
- Chiffré : un presque-multiple de p dont la parité contient le message.
- Sécurité : problème du plus grand commun diviseur approximatif, AGCD.

Le problème AGCD

Sans bruit : attaque immédiate

Si l'on donne des multiples exacts

$$x_i = pq_i,$$

alors un PGCD permet de retrouver p .

Avec bruit : structure masquée

On publie seulement des presque-multiples

$$x_i = pq_i + r_i,$$

où les r_i sont petits. Le PGCD exact disparaît et retrouver p devient difficile.

Exemple AGCD du mémoire

Sans bruit

$$x_1 = 91, \quad x_2 = 143$$

$$\text{pgcd}(91, 143) = 13.$$

La clé $p = 13$ est retrouvée.

Avec bruit

$$x_1 = 13 \cdot 7 + 2 = 93$$

$$x_2 = 13 \cdot 11 + 3 = 146$$

$$\text{pgcd}(93, 146) = 1.$$

Le bruit n'est pas un défaut : c'est ce qui empêche d'extraire la clé.

DGHV symétrique : paramètres

Paramètres du mémoire

Pour un paramètre de sécurité λ :

- p : clé secrète, entier impair, taille environ λ^2 bits ;
- q : grand entier aléatoire, taille environ λ^5 bits ;
- r : petit entier aléatoire, taille environ λ bits ;
- $b \in \{0, 1\}$: bit à chiffrer.

Idée

Construire un entier très proche d'un multiple de p , en plaçant le bit dans la parité.

DGHV symétrique : chiffrement

Algorithme de chiffrement

$$c = qp + 2r + b.$$

- qp cache la structure dans un grand multiple de p .
- $2r$ ajoute du bruit, mais reste pair.
- b fixe la parité du chiffré modulo p .

grand multiple de p + petit bruit pair + bit

DGHV symétrique : déchiffrement

Algorithme de déchiffrement

$$b = (c \bmod p) \bmod 2.$$

$$c \bmod p = (qp + 2r + b) \bmod p = 2r + b.$$

$$(2r + b) \bmod 2 = b.$$

Condition de correction

Le bruit doit rester assez petit :

$$|2r + b| < \frac{p}{2}.$$

DGHV asymétrique : rendre le chiffrement public

Clés

- clé secrète : p ;
- clé publique : des presque-multiples

$$x_i = q_i p + 2r_i.$$

Chiffrement public d'un bit b

On choisit un sous-ensemble aléatoire S et un bruit r :

$$c = \left(b + 2r + \sum_{i \in S} x_i \right) \bmod x_0.$$

Le chiffré garde la forme $pQ + 2R + b$, donc le même déchiffrement fonctionne.

Pourquoi le chiffrement public garde la bonne forme

Développement

Comme $x_i = q_i p + 2r_i$, on a

$$c = b + 2r + \sum_{i \in S} (q_i p + 2r_i) + kx_0.$$

En remplaçant aussi $x_0 = q_0 p + 2r_0$:

$$c = p \left(\sum_{i \in S} q_i + kq_0 \right) + 2 \left(r + \sum_{i \in S} r_i + kr_0 \right) + b.$$

Conclusion

Le serveur chiffre avec la clé publique, mais seul p permet de retirer le multiple principal et de lire la parité.

Addition homomorphe dans DGHV

Deux chiffrés

$$c_1 = pq_1 + 2r_1 + b_1, \quad c_2 = pq_2 + 2r_2 + b_2.$$

Somme

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (b_1 + b_2).$$

- Modulo 2, $b_1 + b_2$ correspond à $b_1 \oplus b_2$.
- Le bruit devient $r_1 + r_2$: croissance linéaire.

Produit

$$c_1 c_2 = (pq_1 + 2r_1 + b_1)(pq_2 + 2r_2 + b_2).$$

En regroupant les multiples de p :

$$c_1 c_2 = p(\dots) + 2(2r_1 r_2 + r_1 b_2 + r_2 b_1) + b_1 b_2.$$

- Le message devient $b_1 b_2$: porte AND.
- Le bruit contient un terme $r_1 r_2$: croissance beaucoup plus rapide.
- C'est la multiplication qui limite la profondeur du circuit.

Budget de bruit : exemple de correction

Clé secrète

Prenons $p = 101$, donc le seuil critique est $p/2 = 50,5$.

Correct

$$b = 1, \quad 2r = 10$$

$$c = 2000 \cdot 101 + 10 + 1.$$

$$c \bmod 101 = 11, \quad 11 \bmod 2 = 1.$$

Échec

$$b = 0, \quad 2r = 64$$

$$64 > 50,5, \quad [c]_{101} = 64 \equiv -37.$$

$$-37 \bmod 2 = 1 \neq 0.$$

DGHV : du somewhat au fully homomorphic

Limite

DGHV sait additionner et multiplier, mais seulement tant que le bruit reste sous le seuil. Il est donc d'abord *somewhat homomorphic*.

Idée de Gentry

Si le schéma peut évaluer son propre déchiffrement, alors il peut produire :

$$c_{\text{bruit}} \longmapsto \text{Enc}(\text{Dec}(c_{\text{bruit}})).$$

C'est le même message, dans un chiffré rafraîchi.

Le serveur reçoit une clé de bootstrapping : la clé secrète elle-même chiffrée.

Pourquoi le bootstrapping DGHV est difficile

Circuit de déchiffrement

$$m = (c \bmod p) \bmod 2.$$

- Le calcul $c \bmod p$ repose sur une division euclidienne.
- Cette division a une profondeur multiplicative trop grande.
- L'évaluer homomorphiquement créerait plus de bruit qu'elle n'en retire.

Solution du mémoire

Alléger le déchiffrement avant de le bootstrapper : c'est la technique du *squashing*.

Squashing : alléger le déchiffrement

Principe

On ajoute à la clé publique des nombres rationnels y_i et on remplace le secret $1/p$ par une approximation utilisant un vecteur très creux :

$$\sum_{i=1}^n s_i y_i \approx \frac{1}{p}, \quad s_i \in \{0, 1\}.$$

Déchiffrement écrasé

Le serveur calcule en clair

$$z_i = (c y_i) \bmod 2,$$

puis le déchiffrement devient

$$b = \left(c - \left\lfloor \sum_{i=1}^n s_i z_i \right\rfloor \right) \bmod 2.$$

Comme le vecteur s est creux, la somme contient peu de termes effectifs.

Ce que DGHV montre très bien

- le bruit protège le secret ;
- addition et multiplication donnent XOR et AND ;
- le bootstrapping transforme un schéma limité en FHE.

Limites

- chiffrés très volumineux ;
- multiplication très coûteuse ;
- bootstrapping complexe à rendre efficace.

Changement de paradigme

CKKS chiffre des nombres réels ou complexes et accepte une erreur contrôlée. Le bruit est interprété comme une approximation numérique.

- Adapté aux statistiques, au traitement du signal et au machine learning.
- Pas adapté aux calculs exacts bit à bit.
- Très intéressant car il peut traiter plusieurs valeurs dans un même chiffré.

exactitude parfaite $\longrightarrow$ précision flottante contrôlée.

Anneau quotient

CKKS travaille dans

$$R_q = \mathbb{Z}_q[X]/(X^N + 1),$$

avec N généralement puissance de 2.

- Les coefficients sont pris modulo q .
- Les polynômes sont réduits modulo $X^N + 1$.

Racines de l'unité

Pour $n \geq 1$:

$$\omega = \exp\left(\frac{i\pi}{n}\right)$$

est une racine $2n$ -ième de l'unité. CKKS utilise les racines primitives pour relier vecteurs et polynômes.

Encodage simplifié

Pour un vecteur $\mathbf{z} = (z_1, \dots, z_{n/2}) \in \mathbb{C}^{n/2}$:

$$\text{Encodage}_{\Delta}(\mathbf{z}) = \lfloor \Delta \sigma(\mathbf{z}) \rfloor \in \mathbb{Z}[X]/(X^N + 1).$$

σ représente l'interpolation polynomiale ; Δ contrôle la précision.

Pourquoi une échelle ?

Les calculs dans R_q utilisent des entiers modulaires. Pour représenter un réel m , on encode :

$$\tilde{m} = \lfloor \Delta m \rfloor.$$

- Plus Δ est grand, plus la précision initiale est bonne.
- Mais les valeurs encodées grandissent et consomment le module q .
- L'arrondi est déjà une première source d'erreur.

Chaîne de traitement

- 1 Générer les clés (pk, sk) dans R_q .
- 2 Encoder un vecteur $\mathbf{z}$ avec l'échelle Δ .
- 3 Chiffrer le polynôme encodé.
- 4 Évaluer additions et multiplications.
- 5 Après chaque multiplication : relinéarisation puis rescaling.
- 6 Déchiffrer et décoder pour obtenir une approximation.

CKKS : génération des clés

Dans $R_q = \mathbb{Z}_q[X]/(X^N + 1)$

- 1 choisir un petit secret s , souvent à coefficients dans $\{-1, 0, 1\}$;
- 2 choisir a aléatoire dans R_q ;
- 3 choisir un petit bruit e ;
- 4 publier

$$\text{pk} = (a, b), \quad b = -as + e \pmod{q}.$$

Sécurité

La paire (a, b) cache s à cause du bruit e .

Message encodé

On part de

$$m = \text{Encodage}_{\Delta}(\mathbf{z}).$$

Chiffrement

On choisit un petit u et deux petits bruits e_1, e_2 , puis

$$c_0 = bu + e_1 + m \pmod{q}, \quad c_1 = au + e_2 \pmod{q}.$$

Le chiffré est

$$\mathbf{c} = (c_0, c_1) \in R_q^2.$$

Calcul de déchiffrement

$$c_0 + c_1 s.$$

En remplaçant b par $-as + e$:

$$\begin{aligned} c_0 + c_1 s &= (bu + e_1 + m) + (au + e_2)s \\ &= m + (eu + e_1 + e_2 s). \end{aligned}$$

Conclusion

On retrouve le message encodé m à une petite erreur près, puis on décode et on divise par l'échelle.

CKKS : addition homomorphe

Deux chiffrés

$$\mathbf{c} = (c_0, c_1), \quad \mathbf{d} = (d_0, d_1).$$

Addition coefficient par coefficient

$$\mathbf{c} + \mathbf{d} = (c_0 + d_0, c_1 + d_1).$$

$$(c_0 + d_0) + (c_1 + d_1)s = (c_0 + c_1s) + (d_0 + d_1s) \approx m + m'.$$

Le bruit augmente peu : il s'additionne.

CKKS : multiplication homomorphe

Produit de deux chiffrés de taille 2

$$(c_0, c_1)(d_0, d_1) = (c_0 d_0, c_0 d_1 + c_1 d_0, c_1 d_1) = (x_0, x_1, x_2).$$

Pourquoi cela correspond au produit

$$x_0 + x_1 s + x_2 s^2 = (c_0 + c_1 s)(d_0 + d_1 s) \approx mm'.$$

- Le résultat est correct algébriquement.
- Mais le chiffré passe de taille 2 à taille 3.
- Il faut relinéariser.

CKKS : relinéarisation

Problème

Après multiplication, le déchiffrement contient un terme en s^2 :

$$m \approx c_0 + c_1 s + c_2 s^2.$$

Clé de relinéarisation

On dispose d'une clé (rk_0, rk_1) qui chiffre s^2 :

$$rk_0 + rk_1 s \approx s^2.$$

Remplacement

$$c_2 s^2 \approx c_2 (rk_0 + rk_1 s),$$

donc

$$(c'_0, c'_1) = (c_0 + c_2 rk_0, c_1 + c_2 rk_1).$$

Problème d'échelle

Si

$$\tilde{m}_1 \approx m_1 \Delta, \quad \tilde{m}_2 \approx m_2 \Delta,$$

alors

$$\tilde{m}_1 \tilde{m}_2 \approx m_1 m_2 \Delta^2.$$

Rescaling

On divise le chiffré par Δ :

$$c_{\text{rescaled}} = \left\lfloor \frac{c}{\Delta} \right\rfloor \approx m_1 m_2 \Delta.$$

Cela garde les valeurs sous contrôle.

CKKS : une multiplication complète

$$\mathbf{c}, \mathbf{d} \xrightarrow{\text{produit}} (x_0, x_1, x_2) \xrightarrow{\text{relinéarisation}} (c'_0, c'_1) \xrightarrow{\text{rescaling}} \mathbf{c}_{\text{propre}}.$$

Ce que chaque étape corrige

- multiplication : calcule le produit des messages ;
- relinéarisation : ramène un chiffré de taille 3 à taille 2 ;
- rescaling : ramène l'échelle de Δ^2 à Δ .

CKKS : bruit, module et stratégie levelled

Paramètres pratiques

Il faut choisir ensemble :

- le degré N ;
- le module q ;
- l'échelle Δ ;
- la profondeur multiplicative attendue.

Levelled FHE

Si le circuit est connu à l'avance, on choisit q assez grand pour supporter tous les rescalings, sans bootstrapping.

Le bootstrapping CKKS existe, mais il est coûteux ; en pratique on l'évite dès que possible.

TFHE : pourquoi l'introduire après DGHV et CKKS ?

Troisième voie

DGHV montre le principe, CKKS optimise le calcul numérique approché, TFHE vise le calcul logique exact avec un bootstrapping très rapide.

- TFHE signifie *Fast Fully Homomorphic Encryption over the Torus*.
- Introduit par Chillotti, Gama, Georgieva et Izabachène.
- Améliore l'idée de FHEW.
- Point fort : évaluer des portes logiques en rafraîchissant le bruit.

Définition

Le tore en dimension 1 est

$$\mathbb{T} = \mathbb{R}/\mathbb{Z}.$$

On représente chaque classe par un réel dans $[0, 1[$.

- Les calculs sont faits modulo 1.
- Cette structure est naturelle pour coder des phases ou des positions circulaires.
- TFHE y place les messages et le bruit.

TLWE : forme d'un chiffré TFHE

Échantillon TLWE

Pour une clé secrète binaire $\mathbf{s} \in \{0, 1\}^k$, un message $\mu \in \mathbb{T}$ est chiffré par

$$(\mathbf{a}, b) \in \mathbb{T}^k \times \mathbb{T}$$

avec

$$b = \sum_{i=1}^k a_i s_i + e + \mu \pmod{1}.$$

- $\mathbf{a}$ est uniforme ;
- e est un petit bruit ;
- la sécurité repose sur la difficulté TLWE.

Encodage des bits dans TFHE

Codage du mémoire

Un bit $m \in \{0, 1\}$ est encodé par

$$\mu = m \cdot \frac{1}{4} \in \mathbb{T}.$$

Bit 0

$$m = 0 \Rightarrow \mu = 0.$$

Bit 1

$$m = 1 \Rightarrow \mu = \frac{1}{4}.$$

Si le bruit dépasse environ $1/8$, les deux valeurs ne sont plus distinguables.

Objectif

Évaluer des portes booléennes directement sur des chiffrés TLWE.

- AND, OR, XOR permettent de construire des circuits complets.
- Chaque porte ajoute du bruit.
- TFHE rend possible un rafraîchissement très fréquent, typiquement à chaque porte.

Différence avec CKKS

CKKS amortit les calculs dans des vecteurs et évite le bootstrapping ; TFHE assume le bootstrapping comme une opération rapide et centrale.

Idée majeure

Le bootstrapping TFHE ne fait pas seulement baisser le bruit : il applique aussi une fonction pendant le rafraîchissement.

chiffré bruité $\xrightarrow{\text{PBS} + \text{table de correspondance}}$ chiffré propre de $f(m)$.

- La fonction f est encodée dans une LUT, *Look-Up Table*.
- Le rafraîchissement devient une étape de calcul utile.
- Cela permet des portes, des seuils, des comparaisons ou des fonctions non linéaires.

Forces

- calcul booléen exact ;
- bootstrapping très rapide ;
- circuits profonds grâce au rafraîchissement fréquent.

Positionnement

TFHE complète CKKS : il est moins naturel pour le calcul numérique massif, mais très adapté aux décisions logiques exactes.

Comparer les trois schémas

	DGHV	CKKS	TFHE
Espace	Entiers	Polynômes, réels/complexes	Tore
Données	Bits	Vecteurs approchés	Bits
Calcul	Exact	Approché	Exact
Sécurité	AGCD	RLWE	TLWE
Point clé	Bruit sur presque-multiples	Échelle, rescaling	PBS et LUT
Force	Compréhension du FHE	Calcul numérique vectorisé	Portes logiques rapides
Limite	Peu efficace en pratique	Approximation, bootstrapping lourd	Moins adapté au numérique flottant

Message final du mémoire

Un même objectif

Rendre possible le calcul sur données privées sans donner accès aux données elles-mêmes.

- DGHV explique le mécanisme fondamental : bruit, seuil, bootstrapping.
- CKKS montre comment accepter l'approximation pour gagner en efficacité sur les données numériques.
- TFHE montre comment rendre le bootstrapping assez rapide pour des circuits booléens profonds.

Conclusion

Le FHE n'est pas un schéma unique, mais une famille de compromis entre exactitude, profondeur, type de données et performance.

Références principales

- C. Gentry, *A fully homomorphic encryption scheme*, 2009.
- M. van Dijk, C. Gentry, S. Halevi, V. Vaikuntanathan, *Fully Homomorphic Encryption over the Integers*, 2010.
- I. Chillotti, N. Gama, M. Georgieva, M. Izabachène, *TFHE : Fast Fully Homomorphic Encryption over the Torus*, 2016/2020.
- J. H. Cheon, A. Kim, M. Kim, Y. Song, *Homomorphic Encryption for Arithmetic of Approximate Numbers*, 2017.

Merci de votre attention

Questions ?